

Worksheet No. - 06

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 2nd

Subject Name: Artificial Intelligence Tools

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 02.04.2026

Subject Code: 25CAP-657

Aim/Overview of the practical: Implement Minimax Algorithm

Objective: The objective of implementing the Minimax Algorithm is to understand how decision-making works in competitive environments such as games. It helps in learning how to evaluate possible moves, predict the opponent's actions, and choose the optimal strategy by minimizing losses and maximizing gains, thereby improving problem-solving and strategic thinking skills.

Procedure/Algorithm/Code

Algorithm

1. Start with the current game state.
2. Generate all possible moves for the current player.
3. If the state is terminal (win/lose/draw), return its score.
4. For the maximizing player, choose the move with the highest value.
5. For the minimizing player, choose the move with the lowest value.
6. Recursively repeat steps 2–5 for all possible moves.
7. Return the best move as the optimal decision.

Code

```
import matplotlib.pyplot as plt
```

```
class GameTree:
```

```
    def __init__(self):
```

```
        self.tree = {
```

```
            'A': ['B', 'C'],
```

```
'B': ['D', 'E'],  
  
'C': ['F', 'G'],  
  
'D': [6, 3],  
  
'E': [2, 7],  
  
'F': [9, 3],  
  
'G': [4, 0]  
  
}
```

```
def is_terminal(self, node):  
  
    return isinstance(self.tree.get(node, node), list) and \  
        all(isinstance(x, (int, float)) for x in self.tree[node])  
  
def get_children(self, node):  
  
    return self.tree.get(node, node)  
  
def minimax(node, depth, maximizing_player, game_tree):  
  
    if game_tree.is_terminal(node):  
  
        print(f'{' ' * depth}Leaf {node}: {game_tree.tree[node]}")  
  
        return max(game_tree.tree[node]) if maximizing_player else  
min(game_tree.tree[node])  
  
    children = game_tree.get_children(node)  
  
    print(f'{' ' * depth}Evaluating {node} ({'Max' if maximizing_player else 'Min'})...")  
  
    if maximizing_player:  
  
        best_value = float('-inf')  
  
        for child in children:  
  
            value = minimax(child, depth + 1, False, game_tree)  
  
            print(f'{' ' * (depth + 1)}Value from {child}: {value}")  
  
            best_value = max(best_value, value)
```

else:

```
best_value = float('inf')
```

```
for child in children:
```

```
    value = minimax(child, depth + 1, True, game_tree)
```

```
    print(f'{' ' * (depth + 1)} Value from {child}: {value}')
```

```
    best_value = min(best_value, value)
```

```
print(f'{' ' * depth} Best value at {node}: {best_value}')
```

```
return best_value
```

```
def visualize_tree():
```

```
    print("\nGame Tree structure:")
```

```
    print("    A    ")
```

```
    print("  /  \  ")
```

```
    print(" B    C  ")
```

```
    print(" /\\  /\\ ")
```

```
    print(" D E F G ")
```

```
    print(" [6,3][2,7][9,3][4,0]")
```

```
def main():
```

```
    game_tree = GameTree()
```

```
    visualize_tree()
```

```
    print("\nRunning Minimax Algorithm:")
```

```
    result = minimax('A', 0, True, game_tree)
```

```
    print("\nFinal Result:")
```

```
    print("Optimal value for root (Max player):", result)
```

```
    print("This represents the best guaranteed outcome assuming both players play optimally.")
```

```
if __name__ == "__main__":
```

```
    main()
```

Output

Game Tree structure:

```
      A
     / \
    B   C
   / \ / \
  D  E F  G
[6,3][2,7][9,3][4,0]
```

Running Minimax Algorithm:

Evaluating A (Max)...

Evaluating B (Min)...

Leaf D: [6, 3]

Value from D: 6

Leaf E: [2, 7]

Value from E: 7

Best value at B: 6

Value from B: 6

Evaluating C (Min)...

Leaf F: [9, 3]

Value from F: 9

Leaf G: [4, 0]

Value from G: 4

Best value at C: 4

Value from C: 4

Best value at A: 6

Final Result:

Optimal value for root (Max player): 6

This represents the best guaranteed outcome assuming both players play optimally.

Learning outcomes (What I have learnt):

1. Understood the concept of the Minimax Algorithm.
2. Learnt how decision-making works in two-player games.
3. Understood maximizing and minimizing strategies.